

```
from tabulate import tabulate
```

```
def knapSack(W, wt, val, n):
```

```
    # Create DP table K[][]
```

```
    K = [[0 for _ in range(W + 1)] for _ in range(n + 1)]
```

```
    # Build table K[][] in bottom-up manner
```

```
    for i in range(n + 1):
```

```
        for w in range(W + 1):
```

```
            if i == 0 or w == 0:
```

```
                K[i][w] = 0
```

```
            elif wt[i - 1] <= w:
```

```
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w])
```

```
            else:
```

```
                K[i][w] = K[i - 1][w]
```

```
    # Store result of knapsack
```

```
    res = K[n][W]
```

```
    w = W
```

```
    selected_items = []
```

```
    # Trace selected items
```

```
    for i in range(n, 0, -1):
```

```
        if res <= 0:
```

```
            break
```

```
        if res == K[i - 1][w]:
```

```
            continue
```

```
        else:
```

```
            selected_items.append(i - 1)
```

```
            res -= val[i - 1]
```

```
            w -= wt[i - 1]
```

```
return K, K[n][W], selected_items
```

```
# ----- MAIN -----
```

```
if __name__ == "__main__":
```

```
    print("\n=== 0/1 Knapsack Problem (Dynamic Programming) ===\n")
```

```
    n = int(input("Enter number of items: "))
```

```
    val = list(map(int, input("Enter values of items (space-separated): ").split()))
```

```
    wt = list(map(int, input("Enter weights of items (space-separated): ").split()))
```

```
    W = int(input("Enter capacity of knapsack: "))
```

```
    if len(val) != n or len(wt) != n:
```

```
        print("\n ❌ Error: Number of values/weights does not match number of items!")
```

```
    else:
```

```
        K, max_value, selected_items = knapSack(W, wt, val, n)
```

```
    print("\n 📄 DP Table:")
```

```
    print(tabulate(K, headers=[f"W={i}" for i in range(W + 1)], tablefmt="grid"))
```

```
    print(f"\n 💰 Maximum Value that can be put in Knapsack = {max_value}")
```

```
    print("\n ✅ Selected Items:\n")
```

```
    output_table = [
```

```
        [i + 1, val[i], wt[i], round(val[i] / wt[i], 2)]
```

```
        for i in selected_items[::-1]
```

```
    ]
```

```
    print(tabulate(output_table, headers=["Item No.", "Value", "Weight", "Value/Weight"],  
tablefmt="grid"))
```

OUTPUT :

=== 0/1 Knapsack Problem (Dynamic Programming) ===

Enter number of items: 2

Enter values of items (space-separated): 100 120

Enter weights of items (space-separated): 10 25

Enter capacity of knapsack: 30